

Property-based testing: Part 1

17-313: Foundations of Software Engineering

<https://cmu-313.github.io>

Michael Hilton and **Josh Sunshine**

Fall 2026

Administrivia

- Problem Set 4 due tomorrow (Friday, Oct 2)
- Midterm Review, Monday October 5 in Recitation
- Quiz, Tuesday October 6
- Midterm October 8
 - Prior-year midterms released to you
 - Questions:
 - Half like prior year midterms
 - Half questions like problem sets and quizzes, *but **free response***
 - No cheat sheets

Learning Goals

- Understand the tradeoffs between property-based testing to example-based testing
- Describe techniques for automatically generating input
- Identify common properties
- Analyze the quality of a test suite

What are the goals of building a test suite? How can you measure if you've achieved each goal?

Test Suite Quality Metric 1: Code coverage

- **Measures** which parts of the code the tests execute
- **Common criteria**, from weakest to strongest:
 - Statement: every line ran at least once
 - Branch: every if went both true and false
 - Condition: each boolean sub-condition independently affects the outcome
 - Path: every route through the code, usually impossible with loops
- **Strengths**: cheap, automatic, and good at revealing code that no test touches at all
- **Low coverage** is informative because untested code is definitely unchecked
- **High coverage is less so**: coverage measures execution, not checking

Test Suite Quality Metric 2: Mutation score

- **Idea:** inject small, deliberate bugs ("mutants") and check whether the suite notices
- **Common mutation operators:**
 - swap relational operators ($< \rightarrow <=$)
 - swap arithmetic operators ($+ \rightarrow -$)
 - negate a condition
 - delete a statement
 - change a constant
- **Run the suite against each mutant:**
 - *killed*: at least one test fails
 - *survived*: every test still passes, which is a gap in the suite
- **Mutation score** = killed mutants $\div$ *non-equivalent* mutants
- **Key assumption (the "coupling effect"):** tests that catch simple faults also tend to catch complex ones

Mutation testing challenges

- **Cost:** one suite run per mutant, and a real codebase produces thousands of mutants
 - common mitigations: run only the tests that cover the mutated line, sample mutants, and mutate only changed code
- **Equivalent mutants** behave identically to the original, so no test can kill them
 - deciding whether a mutant is equivalent is undecidable in general
- **Not every surviving mutant is worth killing**, for example mutants in logging or in defensive code
- **Evidence:** mutant detection correlates with detecting real faults (Just et al., FSE 2014)
- **At scale:** Google shows a few surviving mutants per change during code review rather than reporting a single score

Mutation testing tools

cargo-
mutants



mutmut



Ground-truth measures

- **Idea:** measure what we actually care about, which is whether the suite catches real bugs
- **Historical bug replay:** check out old buggy versions of the code and see whether the suite fails on them
 - Defects4J is the standard research benchmark: hundreds of real Java bugs
 - **Escaped defects:** count bugs found in production or by users that a test should have caught
- **Strength:** no proxy assumptions, and this is how researchers validate coverage and mutation testing
- **Limits:** requires a bug history, arrives too late to guide day-to-day testing, and escaped-defect counts depend on how carefully bugs are reported

Measures of oracle strength

- **Idea:** measure how much the tests *check*, not just what they run
- **Checked coverage:** count only statements that influence an assertion's outcome
- **Pseudo-tested methods:** delete a method's entire body, and if no test fails, the method is pseudo-tested
 - a cheap, extreme form of mutation testing
- **Assertion density:** assertions per test or per line of test code
- **Test smells:**
 - tests with no assertions
 - *assertion roulette*: many unlabeled assertions in one test
 - *eager tests*: one test checking many behaviors at once
- **Limits:** counting assertions says nothing about whether they're the right ones

Input-space measures

- **Idea:** measure which kinds of inputs the tests try, rather than which code they reach
- **Partition and boundary coverage:** does every input class get tested, including its edges?
 - for a list function: empty, one element, duplicates, very large
- **Combinatorial (t-way) coverage:** is every pair (or triple) of configuration options tested together?
 - NIST studies found most configuration-interaction faults involve only a few parameters, so pairwise testing catches a large share of them
- **Requirements coverage:** does every requirement or user story have at least one test?
- **Limits:** partitions are chosen by a human, so the measure is only as good as the analysis behind it

Practical cost measures

- **Idea:** a good suite also has to be usable
- **Flakiness rate:** how often a test fails without any code change
- **Runtime and feedback time:** slow suites get run less often, or skipped
- **Maintenance burden:** how often tests break on intentional changes
- **Diagnosability:** when a test fails, how quickly can a developer find the cause?
- **Limits:** these metrics measure cost, not effectiveness, and a fast, stable suite can still miss everything

Property-based testing to the Rescue?



What is property-based testing?

- Introduced by the Haskell Library QuickCheck in 2000
 - Properties are “testable specifications” written in code
 - Advertised as “lightweight formal methods”
- Properties like assertions are automated oracles
 - Properties in testing code, assertions are in program code
- PBT in other languages:
 - Hypothesis for Python
 - **fast-check for JavaScript**

Property-Based Testing replaces single examples with properties that hold for many inputs

```
function isNonDecreasing(values: number[]): boolean {  
  for (let i = 1; i < values.length; i++) {  
    if (values[i - 1] > values[i]) return false;  
  }  
  return true;  
}
```

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const ys = doubleBogosort(xs);  
  return isNonDecreasing(ys);  
}));
```

↖ We want to test our implementation of an amazing sorting algorithm

Property-Based Testing replaces single examples with properties that hold for many inputs

```
function isNonDecreasing(values: number[]): boolean {  
  for (let i = 1; i < values.length; i++) {  
    if (values[i - 1] > values[i]) return false;  
  }  
  return true;  
}
```

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const ys = doubleBogosort(xs);  
  return isNonDecreasing(ys);  
}));
```

 We use a **generator** to automatically explore inputs to our function under test

Property-Based Testing replaces single examples with properties that hold for many inputs

```
function isNonDecreasing(values: number[]): boolean {  
  for (let i = 1; i < values.length; i++) {  
    if (values[i - 1] > values[i]) return false;  
  }  
  return true;  
}
```

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const ys = doubleBogosort(xs);  
  return isNonDecreasing(ys);  
})));
```

We provide a function that checks that a **property** holds under a given test input, xs

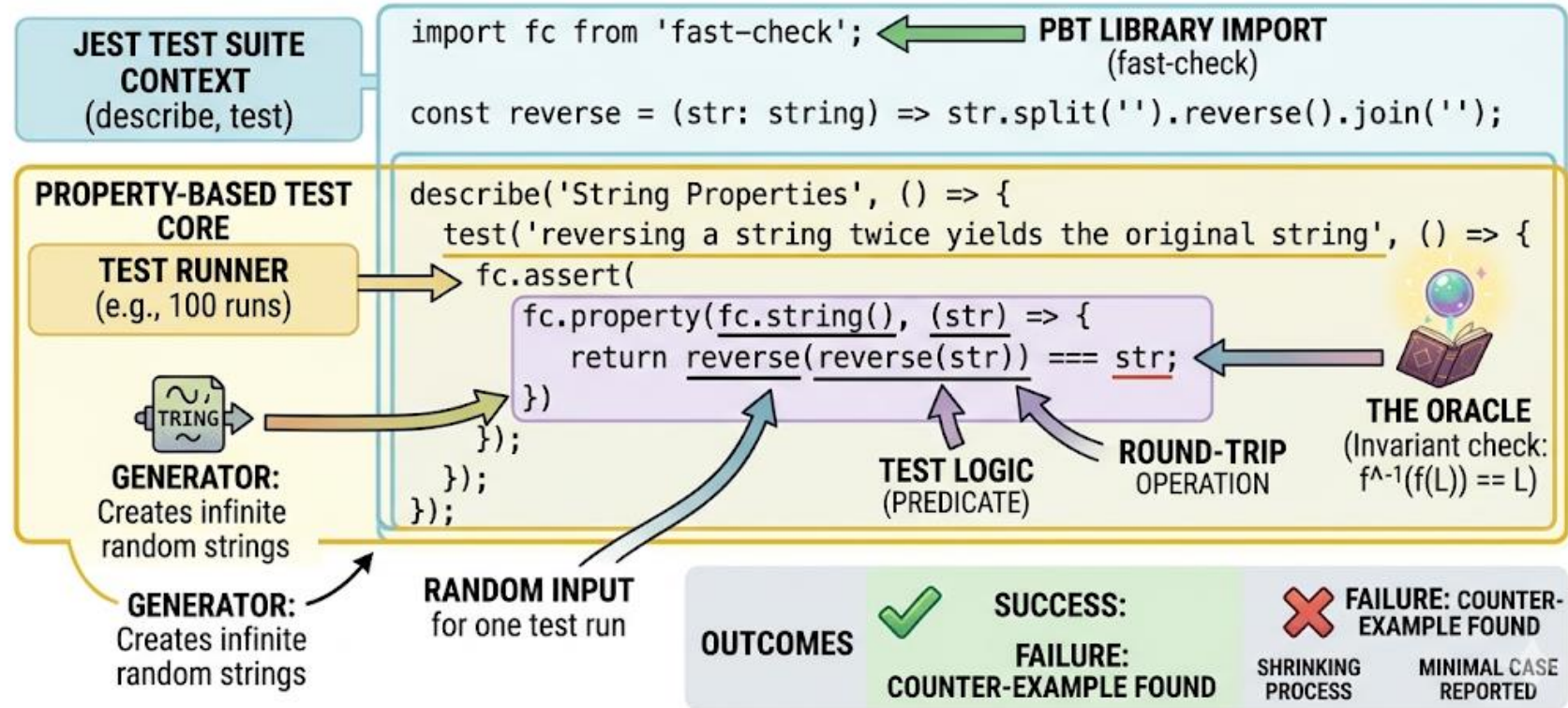
Property-Based Testing replaces single examples with properties that hold for many inputs

```
function isNonDecreasing(values: number[]): boolean {  
  for (let i = 1; i < values.length; i++) {  
    if (values[i - 1] > values[i]) return false;  
  }  
  return true;  
}
```

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const ys = doubleBogosort(xs);  
  return isNonDecreasing(ys);  
}));
```

 In this case, our property checks that array elements are non-decreasing

Anatomy of a Property-based test

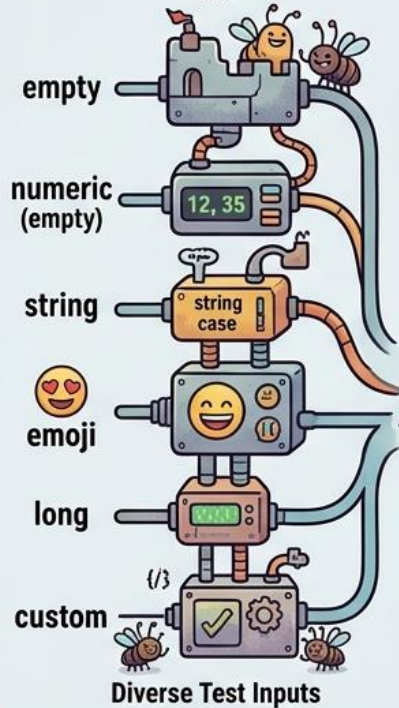


ANATOMY OF A PROPERTY-BASED TEST

The Identity Property: Double Reverse ($\forall \text{ list: reverse(reverse(list))} == \text{list}$)

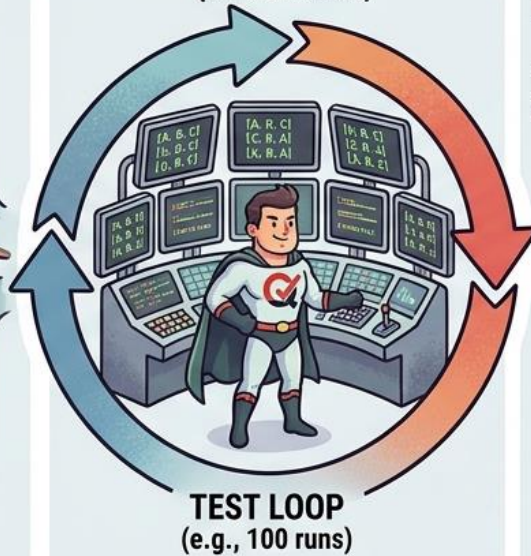
THE GENERATOR

lab



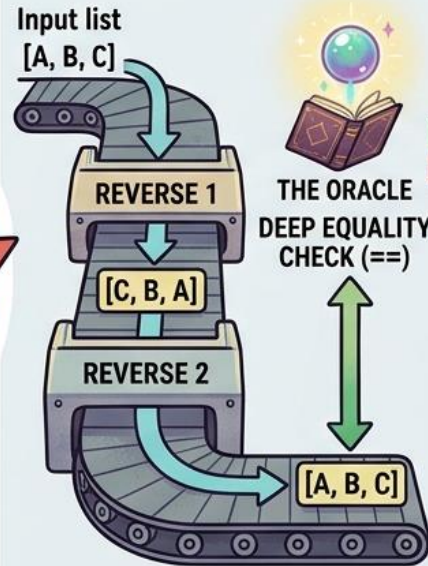
THE GENERATOR

THE PROPERTY RUNNER (control center)



THE PROPERTY RUNNER (control center)

EXECUTION & ORACLE



EXECUTION & ORACLE

OUTCOMES

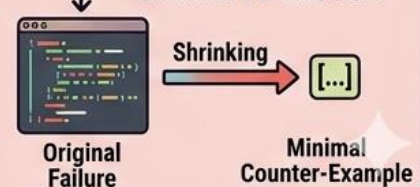
SUCCESS (Tests Pass)



THE SHRINKER



SHRINK TO RESCUE



Common Properties

Common Property: Round Tripping

- If you transform data and then undo it, you should get the original back. Mathematical logic: $f^{-1}(f(x)) = x$

```
fc.assert(  
  fc.property(fc.string(), (str) => {  
    return reverse(reverse(str)) === str;  
  })  
);
```

```
fc.assert(  
  fc.property(fc.string(), (original) => {  
    return decode(encode(original)) === original;  
  })  
);
```

Other common mathematical properties

- **Commutativity** - The order of inputs shouldn't matter for the result. Mathematical logic: $f(a, b) = f(b, a)$
- **Idempotence** - Running the operation multiple times has the same effect as running it once. Mathematical logic: $f(f(x)) = f(x)$
- **Monotonicity** - If the input increases (or stays the same), the output must also increase (or stay the same).
Mathematical logic: *if $x \leq y$ then $f(x) \leq f(y)$*
- **Symmetry** - If Alice is the same as Bob, Bob is the same as Alice.
Mathematical logic: *if $a = b$ then $b = a$*

In-class Exercise

Part 1: Matching Game

Common Properties: Invariants

- Something **stays the same** or **within allowed bounds**
 - Sorting: same multiset of elements (i.e., nothing is added or dropped)
 - Sets and strings: $|AB| = |A| + |B|$
 - Cart totals: never negative
 - Cart: membership tier discount never increases price!

```
fc.assert(fc.property(fc.array(Item), fc.tuple(Tier, Tier), (items, [tLow, tHigh]) => {  
  const order = ["guest", "bronze", "silver", "gold"];  
  const low = computeTotal(items, tLow);  
  const high = computeTotal(items, tHigh);  
  return order.indexOf(tHigh) >= order.indexOf(tLow) ? high <= low : true;  
}));
```

Common Property: Metamorphic

- Property holds under **input transformations**
 - Sorting: reversing/shuffling input does not change the sorted output
 - Cart: reordering items does not change the total
 - JSON parsing: reordering object keys produces the same parsed result
 - Shipping: changing address / ZIP doesn't affect the price

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const a = doubleBogosort(xs);  
  const b = doubleBogosort(xs.reverse());  
  return a.length === b.length && a.every((v,i)=> v === b[i]);  
}));
```

Common Property: Differential

- Two implementations (or versions) **agree on outputs and errors**
 - Slow but trusted reference solution vs. optimized version
 - Old vs. new implementation after refactoring
 - Third-party library vs. your code

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const a = doubleBogosort(xs);  
  const b = radixSort(xs);  
  return a.length === b.length && a.every((v,i)=> v === b[i]);  
}));
```

In-class exercise part 2: Properties in OpenCode or Wordle

Up next ...

- More PBT including “vibe analysis” and more on input-generators on Tuesday.
- Hands on practice with PBT in recitation after Fall break